PeerLearn: Branch Coverage Report

Team: Amogh Shetty (PES1UG24AM034), Aarush Patil (PES1UG24AM005), Abheek Sen (PES1UG24AM009), Allen Anish (PES1UG24AM029). **Course:** Software Engineering, PES University.

Result: 80.4% branch coverage (288 of 358 branches) across the backend (`backend/src`), from the run of all 61 automated tests on commit `cd6df6f`. The same run is detailed in the Unit Test Report.

1. Test run

Item	Value
Date of run	31 Oct 2026, 09:58 IST
Code version	`main` branch, commit `cd6df6f` (github.com/amoghshetty05/peerlearn)
Test framework	Jest 29 with Supertest (HTTP API tests)
Environment	Node.js 22, PostgreSQL 16 test database created fresh for each run
Command	`cd backend && npm test` (runs `jest --runInBand --coverage`)
Automation	The same tests run in GitHub Actions (job `backend`) on every pull request and every push to `main`; the job fails if line coverage drops below 70%

2. Summary

Measure	Result
Test suites	8 passed of 8
Test cases	61 passed, 0 failed, 0 skipped
Total test time	16.5 s
Statement coverage	92.0% (837/910)
Branch coverage	80.4% (288/358)
Function coverage	86.9% (205/236)
Line coverage	94.9% (723/762) (target 70%, NFR-08)

3. Branch coverage by file

Branch coverage counts each possible path through an `if`, `? :`, `||` or `switch` and checks whether the tests took it. Overall, 288 of 358 branches (80.4%) were exercised. Files are listed from lowest to highest branch coverage.

File	Branches covered	Branch coverage
`src/dal/audit.js`	1 of 7	14.3%
`src/app.js`	1 of 2	50.0%
`src/dal/pool.js`	1 of 2	50.0%
`src/dal/reports.js`	2 of 4	50.0%
`src/services/email.js`	3 of 6	50.0%
`src/services/notification.js`	1 of 2	50.0%
`src/dal/sessions.js`	6 of 10	60.0%
`src/errors.js`	3 of 5	60.0%
`src/utils/logger.js`	7 of 11	63.6%
`src/dal/groups.js`	4 of 6	66.7%
`src/utils/time.js`	6 of 9	66.7%
`src/dal/tutors.js`	3 of 4	75.0%
`src/middleware/requestId.js`	3 of 4	75.0%
`src/services/profile.js`	24 of 32	75.0%
`src/schemas.js`	7 of 9	77.8%
`src/middleware/validate.js`	24 of 30	80.0%
`src/serializers.js`	8 of 10	80.0%
`src/services/group.js`	8 of 10	80.0%
`src/dal/users.js`	5 of 6	83.3%
`src/middleware/enforceHttps.js`	5 of 6	83.3%
`src/middleware/rateLimiter.js`	12 of 14	85.7%
`src/services/auth.js`	12 of 14	85.7%
`src/middleware/errorHandler.js`	13 of 15	86.7%

File	Branches covered	Branch coverage
`src/services/moderation.js`	14 of 16	87.5%
`src/config.js`	25 of 28	89.3%
`src/services/session.js`	42 of 47	89.4%
`src/services/scoring.js`	11 of 12	91.7%
`src/dal/notifications.js`	3 of 3	100.0%
`src/dal/requests.js`	0 of 0	100.0%
`src/jobs/scheduler.js`	0 of 0	100.0%
`src/middleware/authenticate.js`	14 of 14	100.0%
`src/middleware/authorize.js`	2 of 2	100.0%
`src/routes/index.js`	4 of 4	100.0%
`src/services/audit.js`	0 of 0	100.0%
`src/services/dashboard.js`	4 of 4	100.0%
`src/services/matching.js`	10 of 10	100.0%
`src/utils/clock.js`	0 of 0	100.0%
Total	**288 of 358**	**80.4%**

Files below 70% branch coverage: 11. The untested branches are mainly optional variations the tests do not need: database queries with and without an optional filter (for example listing audit records for one user or for all users), default values, and settings for a real SMTP server, which the tests replace with a simulated one. Every functional requirement is still covered by at least one passing test case.

4. How to reproduce

1. Start PostgreSQL 16 locally (or use Docker).
2. `cd backend && npm ci`
3. `TEST_ADMIN_URL=postgres://postgres:postgres@localhost:5432/postgres npm test`

In GitHub, open the Actions tab, choose any run of the CI workflow and download the `coverage` artifact for the same HTML coverage report.